Phone sensors and a background task

Step detection, location, and a task that outlives the app

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Week 8

TODAY

What this lab is for

Count a step from raw motion

Turn an accelerometer stream into a step count with a threshold and a debounce window, sampling period stated explicitly.

Make work survive the app closing

Schedule a workmanager task that keeps running after the user closes the app, and read its result back on the next launch.

TIME

2 hours

SUBMIT

Push to your team repo, branch `lab-4`, before next lab

SETUP

Where your code goes

1. Branch the team repository and add the four dependencies:

```
git checkout -b lab-4
flutter pub add sensors_plus geolocator
flutter pub add workmanager shared_preferences
```

2. App code lives in `lib/lab4/`. One file per task keeps the diff readable.
3. Run the app once before changing anything, ideally on a real device.

Prefer a real device

The accelerometer, GPS and background scheduler all behave differently, sometimes not at all, in the emulator. Keep one nearby for this lab.

TASK I

A step counter from raw acceleration

1. Read `accelerometerEventStream`, `samplingPeriod: gameInterval` (about 50 Hz).
2. Compute the magnitude, $\sqrt{x^2 + y^2 + z^2}$.
3. Track a slow baseline, then cross a threshold above it to count a step.
4. Debounce so one footfall cannot count twice, and show the total live.

DONE WHEN

- [ ] The count increases when you shake the phone and stays flat when it is still.
- [ ] The sampling period is a named constant, not the default.
- [ ] A single footfall cannot count twice.

TASK I

Threshold and debounce, in code

```
accelerometerEventStream(
  samplingPeriod: SensorInterval.gameInterval,  // ~50 Hz
).listen((e) {
  final m = sqrt(e.x*e.x + e.y*e.y + e.z*e.z);
  _baseline = 0.9*_baseline + 0.1*m;
  if (!_above && m > _baseline + 1.2 &&
      DateTime.now().difference(_last) >= _minGap) {
    _above = true; steps++; _last = DateTime.now();
  } else if (_above && m < _baseline + 0.4) {
    _above = false;
  }
});
```

`_baseline`, `_above`, `_last` are fields declared once.

TASK II

Location, under the right permission flow

1. Check `isLocationServiceEnabled` first: a granted permission with the radio off returns nothing.
2. Call `checkPermission`, then `requestPermission` only if it comes back denied.
3. If the result is `deniedForever`, open the app settings instead of asking again.
4. Read one position and show latitude, longitude and the accuracy radius in meters.

DONE WHEN

- [ ] The screen shows a position with its accuracy in meters.
- [ ] Denying once, then granting, works with no restart.
- [ ] `deniedForever` opens Settings instead of looping.

TASK II

Declare the permission before you ask

```
<!-- android/app/src/main/AndroidManifest.xml -->
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>

<!-- ios/Runner/Info.plist -->
<key>NSLocationWhenInUseUsageDescription</key>
<string>Shows your position while the lab screen is open.</string>
```

An undeclared Android permission is denied forever, silently. A missing iOS usage string ends the app on its first call.

TASK II

Service, permission, request, deniedForever

```
Future<Position?> readOnce() async {
  if (!await Geolocator.isLocationServiceEnabled()) return null;
  var p = await Geolocator.checkPermission();
  if (p == LocationPermission.denied) {
    p = await Geolocator.requestPermission();
  }
  if (p == LocationPermission.deniedForever) {
    await Geolocator.openAppSettings();
    return null;
  }
  return Geolocator.getCurrentPosition(
    locationSettings: const LocationSettings(accuracy: LocationAccuracy.high));
}
```

TASK II

Precise or approximate, and what changes

Android 12 and later
The permission dialog offers a separate Precise or Approximate choice. Approximate can return a fix with a radius of a kilometer or more.

iOS
Precise Location is a per-app toggle in Settings. An app can request temporary full accuracy for one feature, with its own usage string.

Design for the approximate case first. A feature that only works under a small accuracy radius is broken for a share of your users on both platforms.

TASK III

A task that outlives the app

1. Register a periodic task; the Android floor is 15 minutes.
2. In the callback, append a timestamp and the last step count to a list in `shared_preferences`.
3. Mark the dispatcher `@pragma('vm:entry-point')` and keep it top-level.
4. On iOS the same code runs as background fetch, at the system's discretion.

DONE WHEN

- [ ] Closing the app and waiting shows new timestamps on the next launch.
- [ ] The dispatcher is top-level and marked as an entry point.
- [ ] The screen lists timestamps, not just the latest one.

TASK III

Write the result, then let the task end

```
const _key = 'lab4_log';
@pragma('vm:entry-point')
void callbackDispatcher() {
  Workmanager().executeTask((task, data) async {
    final prefs = await SharedPreferences.getInstance();
    final log = prefs.getStringList(_key) ?? [];
    log.add('${DateTime.now()}: $lastStepCount steps');
    await prefs.setStringList(_key, log);
    return true;
  });
}
Workmanager().initialize(callbackDispatcher);
```

`registerPeriodicTask(...)` elsewhere schedules it, every 15 minutes at best.

TASK III

Register the background modes

```
<!-- android/app/src/main/AndroidManifest.xml -->
<!-- No extra permission: never request a battery exemption. -->
<!-- ios/Runner/Info.plist -->
<key>UIBackgroundModes</key>
<array>
  <string>fetch</string>
  <string>processing</string>
</array>
<key>BGTaskSchedulerPermittedIdentifiers</key>
<array>
  <string>com.yourteam.app.lab4Sync</string>
</array>
```

The plist identifier must match the one workmanager registers.

TASK III

Two platforms, two guarantees

QUESTION	ANDROID	IOS
How often at best	every 15 minutes, the floor	at the system’s discretion
Under Doze, low power	deferred further, sometimes hours	may not run for a while
What you can promise	a run happened, not exactly when	the code path exists, not that it fired

For the acceptance check, leave the app closed for at least fifteen minutes on Android before you look for a new timestamp.

ACCEPTANCE

What the grader will do

- [ ] Close the app, wait past fifteen minutes, reopen it: a list of timestamps has grown while it was closed.
- [ ] Shake the phone: the step count on screen increases, and standing still leaves it flat.
- [ ] Read a location: the screen shows latitude, longitude and an accuracy value in meters.

One screenshot per task is the cheapest proof you can leave. The timestamp list, a shake in progress, and a position with its accuracy, side by side in the PR.

TROUBLESHOOTING

Four failures you will meet

`Sensor stream never fires on the emulator`	The emulated accelerometer sits still by default. Use a real device, or the emulator's virtual sensors panel.
`The permission dialog never appears`	The manifest or the plist is missing the entry. Both must be declared before the dialog can show.
`The workmanager task never runs`	Battery optimization, a non-top-level callback, or a missing `@pragma('vm:entry-point')`.
`The iOS task never runs at all`	The identifier is missing from, or does not match, `BGTaskSchedulerPermittedIdentifiers`.

Push before you leave.

Branch lab-4 · one commit per task · a screenshot of the timestamp list in the PR description